Threat Model Security Panels

Copyright © 2024 by Black Duck. All rights reserved. No part or parts of this documentation may be reproduced, translated, stored in any electronic retrieval system, transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without prior written permission of the copyright owner. Black Duck retains the exclusive title to all intellectual property rights relating to this documentation.

The information in this documentation is subject to change without notice and should not be construed as a commitment by Black Duck. Black Duck makes no representations or warranties, express or implied, with respect to the documentation and shall not be liable for any damages, including any indirect, incidental, consequential damages (such as loss of profit, loss of use of assets, loss of business opportunity, loss of data or claims for or on behalf of user's customers), that may be suffered by the user.

Black Duck and the Black Duck logo are trademarks of Black Duck. Other brands and products are trademarks of their respective owner(s).

Black Duck Inc.
800 District Ave. Ste 201
Burlington, MA 01803
https://www.blackduck.com/

Contents

Security Panels **1**
- **Mobile Applications** **1**
- **Usernames and Passwords** **2**
- **API Security** **4**
- **OAuth** **5**
- **Data Management** **7**

© 2024 Black Duck

BLACKDUCK®

Security Panels

Mobile Applications

Security Domain	Security Subdomain	Question	Control Evaluation
Message Security	Authentication	How does the application authenticate its users?	Credentials must be verified by server-side components.
	Authorization	How does the application determine the permissions the user has?	The user permission must not be decided based on any information that is presented by the client application to the server application. The entitlement information should be stored at the server-side.
	Transport security	Is the client to server communication over a secure channel?	Data should be encrypted using a secure version of TLS or IPSec.
Data Security	Sensitive data on device	Does the application store sensitive data such as PII, user credentials or cryptographic keys?	Ensure secrets are stored in the keychain (if using iOS) or via PBKDF2 if using Android.
	Device logs	Is sensitive data logged by the application?	Sensitive data should never be persisted to the logs.
	Background resume	Does the application mask data when being moved to the background?	iOS: Prevent the application from entering the background state or mark sensitive UI data beforehand.
			Android: Use the FLAG_SECURE attribute.
Platform and System Software Security	Jailbreak detection	Does the application check whether the device is rooted or jailbroken?	The application should detect and respond to the presence of a rooted or jailbroken device by alerting the user or terminating the application.
	Inter-process communication	How does communication between the application and other applications installed on the device occur?	iOS: Use universal links to receive communication from third-party applications.
			Android: Check access permission of each component to ensure that the access is restricted to a limited set of components

© 2024 Black Duck

Usernames and Passwords

Security Domains	Security Subdomains	Questions	Best Practices
Password policy: Password complexity and related security best practices	Password length and complexity	Are there any password complexity requirements? If yes, specify minimum length and complexity details.	The Password length should be a minimum of 12 characters and a combination of any 3 of the following: Numbers Uppercase letters Lowercase letters Special characters
Credential transmission: Security best practices for credential (username and password) transmission	Transport Layer Security	Does the application use Transport Layer Security to encrypt HTTP traffic? If yes, specify encryption protocol details.	TLSv1.2 or above should be used to encrypt the communication channel between a browser and the server.
Credential transmission: Security best practices for credential (username and password) transmission	HTTP Method	Which HTTP method is used in the request to send credentials to server?	POST HTTP Method must be used to send a login request.
Password storage: Security best practices for password storage	Salting and Hashing	How are passwords stored on the server-side? Specify whether in plain-text, encrypted, hashed or salted hash.	The security best practices include: Use Argon2id with a minimum configuration of 19 MiB of memory, an iteration count of 2, and 1 degree of parallelism If FIPS-140 compliance is required, use PBKDF2 with a work factor of 600,000 or more and set with an internal hash function of HMAC-SHA-256.
Resistance to credential leakage: Security best practices for preventing credentials leakage	Masking	Does the application implement password masking on the client side while entering them?	Password entry should be obscured on the user's screen. The user should be given a choice to either temporarily view the entire masked password, or temporarily view the last typed character of the password on platforms that do not have this as native functionality.
Resistance to credential leakage: Security best practices for preventing credentials leakage	Auto-complete	Does the application have HTML auto-complete enabled?	Auto-complete must be disabled explicitly on the login page for the username and password fields.

© 2024 Black Duck

Security Domains	Security Subdomains	Questions	Best Practices
Resistance to credential leakage: Security best practices for preventing credentials leakage	Remember me	Does the application implement "Remember me" functionality to allow users to maintain session for long duration?	The application should not provide a functionality to enable users to maintain session for long duration without re-login. This functionality is implemented by storing long-term cookies on the browser thereby increasing the attack surface.
Password modification: Security best practices for password modification	Password change	Does the application implement functionality to change password after login? If yes, is the current password required on the change password screen?	Current password should be required to change the password.
Password modification: Security best practices for password modification	Password reset	Please specify controls in place to secure the password reset mechanism?	Password reset functionality as well as any other recovery paths should never reveal the current password. The new password should not be sent in clear text to the user.
User account provisioning: Security best practices to create and remove users	Default password	Does the application manage its own users? If yes, when a user is provisioned, how is the initial password generated and communicated to the user?	The initial password created for new users should be an OTP. The OTP should adhere to best practices as outlined here (for dynamic MFA). When users use the OTP and login, they should be required to change the initial password.
User account provisioning: Security best practices to create and remove users	Default privileges/permissions	What permissions are assigned by default when a new user is added to the application?	Default permissions assigned automatically should be limited in nature and be commensurate with the application functionality.

© 2024 Black Duck

BLACKDUCK®

API Security

Security Domains	Security Subdomains	Questions	Best Practices
Network communication: Best practices for network communication	Transport Layer Security (TLS)	How is the communication between the client and the API endpoint secured? Does it use TLS? Can this API endpoint be also accessed over HTTP (without TLS)? What cipher suites are enabled at the TLS endpoint?	The API should only be accessible over TLS1.2 or higher. It should use only strong cipher suites. https://owasp.org/www-project-cheat-sheets/cheatsheets/TLS_Cipher_String_Cheat_Sheet.
Network communication: Best practices for network communication	Rate-limiting	Is there any rate-limiting implemented on the externally facing APIs? If yes, how is the rate-limiting control implemented?	Rate limiting should be enforced as a control against denial of service or API abuse attacks. When using an API gateway, this control is generally implemented using the API gateway. It is also possible to use other mechanisms to implement this control such as a WAF. It needs to be pointed out that rate limiting makes sense for endpoints that are exposed to externals: businesses and people. Internal APIs need not implement rate limiting.
Network communication: Best practices for network communication	IP restriction	Can IP restriction be enforced for accessing the API endpoint?	If there is a limited number of clients for this API, such as a few business entities accessing this API, then it is recommended to allowlist the IPs that can access this API.
Network communication: Best practices for network communication	HTTP methods	Do you restrict HTTP methods for your API calls?	API endpoints should explicitly say what HTTP methods are supported between GET/POST/PUT/etc.
Input validation: The content received by the API should be properly validated and sanitized before use	Input validation	When processing API requests, what input validation checks are performed on the server-side?	Server-side validation of input data is important. Here we are looking for general input validation on the server-side that validates API parameters using best practices for the input type (e.g., XML, JSON, etc.)

© 2024 Black Duck

OAuth

Security Domains	Security Subdomains	Questions	Best Practices
Grant type: OAuth 2.0 supports four grant types. We should ensure that correct grant types are used.	Grant type	OAuth 2.0 supports four grant types: Auth code grant Implicit grant (this should not be used) Resource owner password grant, and Client credentials grant Which grant type is being used?	A correct grant should be used. If the OAuth Client is: Implicit Grant should not be used. An application executing on a web server, use the Authorization Code Grant. A single page application, use the authorization code grant without client credentials. A native or mobile application, use the Authorization Code Grant with PKCE. A client that is absolutely trusted with user credentials (i.e., the Facebook mobile application accessing Facebook), use the Resource Owner Password Grant. A client that is the owner of the data, use the Client Credentials Grant.
Access token security: OAuth 2.0 Access Tokens should adhere to security best practices.	Scope	Are access tokens scoped commensurate with the business needs?	An access token grants OAuth Clients access to the resource server. The scope of an access token determines what resources and what operations on the resources are granted to the bearer of an access token. Access tokens should be scoped in compliance with need-to-know and least-privilege security principles.
Access token security: OAuth 2.0 Access Tokens should adhere to security best practices.	Expiration	Do the access tokens expire? If yes, what is the expiration timeout?	Access should generally be short-lived. We should ensure that access tokens are reasonably short-lived commensurate with application needs.

© 2024 Black Duck

Security Domains	Security Subdomains	Questions	Best Practices
Access token security: OAuth 2.0 Access Tokens should adhere to security best practices.	Entropy	Are access tokens opaque tokens? If yes, provide how the access tokens are generated.	Opaque access tokens should adhere to the following two security best practices: Opaque access tokens should be at least 128-bits in length. 2. Opaque access tokens should be generated using secure random processes (CSPRNG)
Refresh token security: OAuth 2.0 Refresh Tokens should adhere to security best practices.	Refresh token storage	On the OAUTH Client (e.g., a mobile application), how and where are the refresh tokens stored?	Refresh tokens are stored at OAuth 2.0 Client. If the Client is a mobile client, refresh tokens are stored on the mobile device. These need to be stored securely on the mobile client. Refer to how to store secrets securely on Android devices and on iOS devices.
Authorization code security: OAuth 2.0 Authorization code (used only in the Auth Code flow) should adhere to security best practices.	Complexity	What is the length and complexity/entropy of the authorization codes used?	The authorization codes should be at least 128 bits in length and should be generated using pseudo-random number generator (CSPRNG).
Authorization code security: OAuth 2.0 Authorization code (used only in the Auth Code flow) should adhere to security best practices.	Expiration	How long is the authorization code valid for?	Expiration time for an authorization code should be short. The expiration time of a few minutes is a general norm and anything exceeding 11 mins should be re-evaluated.
Authorization code security: OAuth 2.0 Authorization code (used only in the Auth Code flow) should adhere to security best practices.	Re-usability	Can authorization codes be reused multiple times?	Ensure that the authorization codes are single use only.

© 2024 Black Duck

Security Domains	Security Subdomains	Questions	Best Practices
Miscellaneous: Best practices pertaining to miscellaneous issues such as URI redirect, state parameter, and so on.	Storage at OAuth Client	How is the Client Secret stored at the OAuth 2.0 Client?	We want to ensure that the Client Secret is stored securely on the OAuth 2.0 Client. This question is only pertinent if the OAuth 2.0 Client is a web server. This is not pertinent if the OAuth 2.0 Client is a mobile application.

Data Management

Security Domains	Security Subdomains	Questions	Best Practices
Data protection: Data storage	Server	Is the sensitive data at rest protected using encryption? If, yes please specify what encryption is used.	When encrypted, sensitive data should be encrypted using the encryption algorithm of AES with key length of 128-bits or stronger
Data protection: Data storage	Server	Is the data stored at rest retained for any specific time period? If yes, please specify the time duration.	Retention period of the application data should commensurate with application functionality, compliance needs, and company data retention policies.
Data protection: Data storage	Client	Is the sensitive data stored in the HTML5 localStorage protected using encryption? If, yes, please specify what encryption is used.	When encrypted, sensitive data in localStorage should be encrypted using the encryption algorithm of AES with key length of 128-bits or stronger.
Data handling: Data query	Relational DB	For querying relational databases, what controls are in place to prevent SQL injection vulnerabilities?	To prevent SQL injection, parameterized SQL queries should be used. Even with ORM (objection relational mapping) frameworks, there is a risk of injection attacks because some ORM frameworks support the use of a query language. Even in such cases, parameterized queries should be used.

© 2024 Black Duck

Security Domains	Security Subdomains	Questions	Best Practices
Data handling: Data query	File System	If sensitive data is stored in files, what access controls are present to secure file systems against unauthorized access?	If sensitive data is stored in files on a server, ensure that the access to the files follows the security principle of least-privilege and need-to-know.
Data handling: Data input	Encoding	Is any form of encoding or escaping performed by the application on the data before it is sent to browsers or other output channels? If yes, please elaborate how that is accomplished.	Context specific output encoding should be performed. Usually, the security best practice dictates that such encoding needs to occur in a systematic and institutional way, preferably without and outside of developer control (e.g., use of frameworks to automatically encode messages going to browsers).
Data handling: Data input	Validation	Is any form of validation performed by the application on the input data? If yes, please describe what type of input validation is performed.	Security best practices dictate that input validation such as semantic, syntactic and schematic should be performed on the input data. This validation should be performed on the server end. Ideally, such validation should be performed using systematic features such as input validation libraries.
Data handling: Data input	XML Parsing	Does the application perform any XML data parsing? If yes, please specify how the parsing process is secured to prevent commonly occurring attacks such as XML injection or XML Entities Expansion.	Parser configuration should be hardened to prevent XML entity expansion vulnerability. In addition, input validation should be performed (such as schema checks) to prevent XML injection vulnerabilities.
Data operations: Data backup	Data backup	Is there a provision for data backup? If yes, please elaborate on the frequency of backup.	Periodic backup of application data should be performed.
Data operations: Data backup	Data backup	Are data backups encrypted?	Data backups should be encrypted. Encryption should use AES with 128-bit key length or more.

© 2024 Black Duck